

Wagmann Generals

Professor McCann

CS490

24 March 2026

Engineering Coding Standards Document

1. Purpose and Scope

Define consistent coding practices across the team to ensure:

- Readability and maintainability
- Predictable structure for human and AI contributors
- Reduced bugs and easier debugging
- Scalable codebase growth

Applies to:

- All frontend, backend, and shared code environments
- All contributors (Developers, AI - Code generators)

2. Naming Conventions

a. Variables and Functions

- Use CamelCase (JS/TS/Python)
- Use descriptive names, no abbreviations unless standard

//  Good

```
const userProfile = getUserProfile()
```

//  Bad

```
const up = getUP()
```

b. Classes and Components

- i. Use PascalCase:

```
class UserService {}  
  
function LoginPage() {}
```

c. Constants

- i. Use UPPER_SNAKE_CASE

```
Const MAX_RETRY_COUNT = 3
```

d. Files and Folders

- i. Use kebab-case for files
- ii. Use plural nouns for collections

[user-service.ts](#)

auth-controller.ts

e. API Routes

- i. RESTful, lowercase, plural

GET /users

POST /orders

3. Folder Structure**a. Backend Example:**

src/

|- Controllers/

|- Services/

- | - Repositories/
- | - Models/
- | - Routes/
- | - Middleware/
- | - Utils/
- | - Config/
- | - Tests/

Rules:

- Controllers -> request/response handling only
- Services -> business logic
- Repositories -> database access
- No business logic in controllers

b. Frontend Example

src/

|

| — components/

| — pages/

| — hooks/

| — services/

| — store/

| — utils/

|— styles/

└— assets/

Rules:

- Components = reusable UI
- Pages = route-level views
- Services = API Calls only

4. Code Style (Linting and Formatting)

a. Tools

- Formatter: Prettier
- Linter: ESLint (or Equivalent)

b. Rules

- 1 Tab indentation
- Max Line Length: 100-120 chars
- Always use semicolons (or explicitly decide otherwise)

c. Enforcement

- Pre-commit hooks (e.g., Husky)
- CI must fail on lint errors

5. Error Handling Standards

a. Backend:

- Never return raw errors
- Use structured error responses

```
{  
  "success": false,  
  "error": {  
    "code": "USER_NOT_FOUND",  
    "message": "User does not exist"  
  }  
}
```

b. Error Rules

- Always include:
 - Code (machine-readable)
 - Message (human-readable)
- No stack traces in production responses
- Log full errors internally

c. Try/Catch usage

```
try {  
  // logic  
} catch (error) {  
  logger.error(error)  
  throw new AppError("INTERNAL_ERROR", "Something went wrong")  
}
```

6. API Response Conventions

a. Standard Response Format

- **Success:**

```
{  
    "success": true,  
    "data": {...}  
}
```

- **Error:**

```
{  
    "success": false,  
    "error": {  
        "code": "ERROR_CODE",  
        "message": "Description"  
    }  
}
```

- **Pagination:**

```
{  
    "success": true,  
    "data": [...],  
    "meta": {  
        "page": 1,  
        "limit": 10,  
        "total": 100  
    }  
}
```

7. Logging Standards

a. Use structured Logging

b. Levels:

i. Info

- ii. Warn
 - iii. Error
- c. Never log sensitive data